WP 08. Integrated Circuit Timing and Leakage Corner Sweeps

RESEARCH STATUS	Proposed study. No experimental results have been claimed.
CLUSTER	Semiconductor Manufacturing
EXCEL CROSSWALK	Not explicitly listed
CLOSEST PROJECT	paceval_Industrial_Edge_Neural_Formula_Research_Expanded

Abstract

EDA parameter exploration repeats nonlinear cell delay and leakage expressions over many voltage-temperature-load corners. This study defines a falsifiable and independently testable mathematical kernel for the paceval. Mathematical Engine. The workload and the evaluation criteria below are experimental plans rather than measurements.

1. Industrial scope and relationship to existing research

Distinctive contribution: Cell-level interpretable delay and leakage corner functions, not learned neural formulas.

Workbook relationship: Not explicitly listed. The workbook index and potential commercial benefit are subjective estimates, not measured commercial evidence.

2. Candidate mathematical kernels

Kernel 1

```
Delay: tpd=t0+aC*Cload+aS*Slew+aT*(T-Tref)+aV*(Vref/Vdd)^p
```

Kernel 2

```
Leakage: Ileak=I0*exp(betaT*(T-Tref)+betaV*(Vdd-Vref))
```

Kernel 3

```
Timing slack: slack=Trequired-Tarrival-tpd
```

Kernel 4

```
Power: Pleak=Vdd*Ileak
```

Model expressions are mathematical specifications, not syntax-checked SDK strings. Implementation must declare units, parameter domains, formula IDs, regularization, model assumptions and reference tests.

3. How paceval is actually used

The host application owns parsing and dimensional validation of inputs, formula versioning, candidate generation, dataset provenance, graph-handle lifetime, independent solvers, aggregation and output. paceval owns every supported explicit scalar or composite mathematical expression and its repeated evaluation. Expression creation is performed once per stable model version. The host reuses each computation handle for all compatible input rows. Batch evaluation is used only where it preserves semantics and actually improves measured throughput. The selected paceval SDK or MCP interface must be identified in the experiment log; a local SDK call is not reported as a remote MCP benchmark.

C++ orchestrates candidate enumeration, fitted coefficients, domain-specific numerical solving, independent checking and I/O. paceval should own all supported scalar, composite cost, sensitivity and constraint formulas. Stable models use persistent computation handles and batch execution when beneficial.

4. Proposed numerical experiment and comparators

Screen 100 libraries of synthetic cell families against 1e5 corner combinations per family; keep timing-path aggregation in the host.

Baselines: Compare to vectorized C++, lookup-table interpolation, and actual Liberty library tables if licensing permits. Separate arithmetic time from graph traversal.

Collect median, p95 and p99 latency across repeated runs; throughput, peak RAM, graph creation time, time per reused evaluation, energy where measured and complete pipeline wall time. Record hardware, compiler, precision, parallelism, batching, runtime version, warmup, data layout and input-sequence checksum. Use identical mathematical expressions, tolerances and unit handling in every baseline. Baseline alternatives should include optimized native C++ and, where domain-relevant, specialist solvers. Invalidate claims of engine-wide speedup if the measured advantage exists only in a small formula stage.

5. Independent validation and edge cases

1. Compare to hand-calculated table points and interpolation error.

2. Test Vdd>0, feasible corners and out-of-range extrapolation controls.

3. Verify that the cell delay surrogate is not used as a substitute for static timing analysis.

Predeclare the accepted absolute, relative and application-scaled errors. Verify each expression against an independent implementation using the same inputs and conventions. Include singular, limiting, invalid and near-boundary cases. If a formal interval claim is made, demonstrate coverage of an entire input interval using a method that actually bounds the interval; pointwise TINC checks alone are not such a proof.

6. Go or no-go assessment

A pilot is a technical go only if it passes the independently stated numerical and application acceptance tests and demonstrates either a repeatable improvement of at least 15% in full-pipeline latency or energy under matched conditions, or at least 25% reduction in measured formula-update/integration effort without a performance or quality regression. These are proposed decision thresholds, not measured outcomes. A result below the threshold is a no-go for the claimed benefit, even if the mathematical engine functions correctly.

Principal risk: Actual gate delay depends on waveform shape, load and library characterization. No foundry signoff or timing signoff is implied.

7. Required release artifacts

Provide C++ sources, runtime version, formula manifest, dataset or synthetic generator with fixed seeds, native and high-precision references, failed-case logs, confidence intervals for timings and complete pipeline resource measurements. Never claim deployment, customer validation or certification without separate evidence.

8. Selected external references

1. https://github.com/The-OpenROAD-Project/OpenSTA

2. https://github.com/The-OpenROAD-Project/OpenSTA/blob/master/doc/Examples.md

paceval architecture: https://paceval.com/developers/

Context sources do not certify any paceval results.